SMARTFARM

Documentation technique

Documentation du projet scolaire

Architecture, modèles, routes et fonctionnement interne.

Document préparé à partir du fichier Markdown du projet.

1. Architecture Django

SmartFarm est un projet Django local utilisant Python 3.12, Django 5.2, Django REST Framework 3.16 et SQLite. Le projet Django se trouve dans `config/`; les applications métier sont sous `apps/`. Les routes HTML sont définies par application et incluses dans `config/urls.py`. Les endpoints REST sont protégés par token et répondent en JSON.

Applications

Application	Modules principaux	Responsabilité
`accounts`	`views.py`, `urls.py`	Session web et endpoints d'émission, profil et révocation de token.
`facilities`	`models.py`, `serializers.py`, `api.py`, `views.py`, `services.py`	Bâtiments, espaces et archivage quand des données sont liées.
`sensors`	`models.py`, `serializers.py`, `api.py`, `measurements_api.py`, `measurement_filters.py`, vues HTML	Capteurs, mesures, validation, filtres, tri et pagination.
`actuators`	`models.py`, `serializers.py`, `api.py`, `views.py`	Actionneurs, commandes simulées et historiques.
`dashboard`	`data.py`, `views.py`, `api.py`	Agrégations de lecture pour la page d'accueil et l'API dashboard.
`simulator`	`runtime.py`, `views.py`	Génération des mesures et tâche locale contrôlée par utilisateur.

`alerts` n'est pas une application installée ni développée.

2. Modèles et relations

Modèle	Relations et rôle
`Building`	Entité bâtiment; possède plusieurs `Space`.
`Space`	FK vers `Building` avec `PROTECT`; regroupe capteurs et actionneurs.
`Sensor`	FK vers `Space` avec `PROTECT`; type température ou humidité, unité calculée, seuils minimum/maximum, état actif.
`Measurement`	FK vers `Sensor` avec `PROTECT`; valeur, date mesurée, date reçue et date créée.
`Actuator`	FK vers `Space` avec `PROTECT`; type d'équipement, description, état actif.
`ActuatorCommand`	FK vers `Actuator` avec `PROTECT`; type `POWER_STATE`, valeur `ON`/`OFF`, statut et horodatages.

Les limites techniques des mesures sont −50..80 °C et 0..100 %. Les seuils configurés sur les capteurs sont indépendants des limites physiques. Les commandes et mesures ne disposent d'aucun endpoint de suppression. Les suppressions d'entités liées passent par `archive_or_delete` et désactivent l'entité afin de conserver son historique.

3. Fonctionnement des vues et formulaires

- Les pages HTML des applications facilities, sensors, actuators, dashboard et simulator sont protégées par la session Django (`login_required` ou `LoginRequiredMixin`).
- Les formulaires Django valident les données côté serveur; les attributs HTML et le JavaScript fournissent une validation complémentaire côté client.

- Les pages de suppression des bâtiments, espaces, capteurs et actionneurs demandent une confirmation POST. Une confirmation absente n'entraîne pas de suppression.
- Le dashboard lit les compteurs et mesures à partir des modèles. Le graphique inline reçoit ses points sérialisés par `json_script`, puis dessine un SVG avec le JavaScript du projet.
- Le simulateur affiche les bâtiments, les espaces et les capteurs actifs. Le client ne transmet que les identifiants sélectionnés, l'intervalle et les identifiants du bâtiment/espace; les relations sont revérifiées côté serveur.

4. Serializers et API

DRF utilise `TokenAuthentication`, `IsAuthenticated` par défaut et le renderer JSON. Une réponse JSON 404 dédiée couvre les chemins inconnus sous `/api/v1/`.

Authentification

Méthode / route	Résultat
`POST /api/v1/auth/token/`	Vérifie les identifiants Django et retourne le token.
`GET /api/v1/auth/me/`	Retourne l'identifiant et le nom de l'utilisateur authentifié.
`POST /api/v1/auth/token/revoke/`	Supprime uniquement le token présenté.

Bâtiments et espaces

Les `BuildingSerializer` et `SpaceSerializer` valident les noms et relations. Les viewsets REST complets exposent `GET`, `POST`, `PUT`, `PATCH` et `DELETE` sur les collections/détails selon la sémantique REST; les actions `POST .../<id>/archive/` permettent une désactivation explicite. `GET /api/v1/spaces/?building=<id>` filtre par bâtiment.

Capteurs et mesures

`SensorSerializer` valide le type, l'espace, les seuils et leur plage technique; l'unité est calculée (`°C` ou `%`). Le viewset des capteurs expose le CRUD et les filtres `sensor_type`, `space`, `building`, `is_active`. Son action historique `GET /api/v1/sensors/<id>/measurements/` est paginée.

`MeasurementSerializer` vérifie le capteur actif et sa hiérarchie active, le caractère numérique fini, la précision décimale et les limites physiques. `MeasurementViewSet` expose seulement `GET` liste/détail et `POST` création : pas d'édition/suppression des mesures. Les filtres sont `sensor`, `space`, `building`, `date_from`, `date_to`; le tri accepte `recorded_at` et `-recorded_at`. Les listes utilisent `MeasurementPagination` : 20 par défaut, `page_size` configurable, maximum 100.

Actionneurs et commandes

`ActuatorSerializer` valide le nom, le type et l'espace actif. Le viewset d'actionneurs est archivable et filtre par `actuator_type`, `building`, `space` et `is_active`. `ActuatorCommandSerializer` n'accepte pas de statut ou date d'exécution fournis par le client; il crée les commandes au statut `PENDING`. Le type de commande autorisé est `POWER_STATE`, les valeurs sont `ON` ou `OFF`. Les commandes sont refusées si l'actionneur, l'espace ou le bâtiment est inactif. L'API propose la liste paginée, la création et le détail, sans modification/suppression.

Dashboard

Les endpoints `GET /api/v1/dashboard/summary/`, `environment/`, `measurements/` et `actuators/` retournent respectivement les compteurs, la dernière mesure par type, les mesures récentes et la liste des actionneurs avec leur dernière commande. Le endpoint dashboard des mesures accepte `period=24h` ou `period=7d` pour ses séries de graphique. Aucun appel ne crée de donnée.

Simulateur

Route	Méthode	Fonction
`/simulator/spaces/?building_id=<id>`	GET	Retourne les espaces actifs du bâtiment.
`/simulator/start/`	POST	Valide puis démarre la tâche du compte courant; réponse 202, 400 en entrée invalide, 409 si déjà active.
`/simulator/stop/`	POST	Signale l'arrêt et attend la fin de la requête HTTP en vol; réponse 200.
`/simulator/status/`	GET	Retourne l'état, les compteurs, dernières valeurs et erreurs de l'utilisateur.

Les actions de la page web utilisent `SessionAuthentication` et CSRF. `runtime.py` détient le token DRF côté serveur et appelle `POST /api/v1/measurements/` via `urllib`, timeout de cinq secondes. Le thread génère une mesure par capteur à chaque cycle, puis attend l'intervalle avec un `threading.Event`. Un `threading.Lock` protège le registre et les compteurs; l'arrêt pose l'événement puis joint le thread avant de répondre.

Les valeurs normales proviennent de variables d'environnement et sont bornées aux plages physiques. Les erreurs HTTP/réseau sont conservées en mémoire (dix dernières), sans interrompre automatiquement les cycles suivants. Le registre est local au processus, non persistant; cette approche est limitée au serveur de développement avec un seul processus.

5. Routes HTML

Groupe	Routes principales
Connexion	`/accounts/login/`, `/accounts/logout/` (POST/CSRF).
Dashboard	`/`.
Bâtiments	`/buildings/`, `/buildings/create/`, `/buildings/<id>/`, `/edit/`, `/delete/`.
Espaces	`/spaces/`, `/spaces/create/`, `/spaces/<id>/`, `/edit/`, `/delete/`.
Capteurs	`/sensors/`, `/sensors/create/`, `/sensors/<id>/`, `/edit/`, `/delete/`.
Mesures	`/measurements/`, `/measurements/<id>/`, `/sensors/<id>/measurements/`.
Actionneurs	`/actuators/`, `/actuators/create/`, `/actuators/<id>/`, `/edit/`, `/delete/`, `/actuators/<id>/commands/`, `/actuators/<id>/commands/create/`, `/commands/`.
Simulateur	`/simulator/`.

6. Erreurs, sécurité et conservation

- Les validations DRF retournent le code 400 et des champs/messages JSON.
- Les ressources absentes retournent 404; une URL API inconnue retourne `{"detail": ...}` en JSON.
- Les endpoints métier demandent un token. Les pages HTML demandent une session.
- Les actions HTML POST sont protégées par CSRF; le simulateur utilise le token CSRF présent dans la page pour ses actions AJAX.
- Les identifiants d'espaces/capteurs du simulateur sont revérifiés et ne suffisent pas à injecter une valeur arbitraire.
- Les FK historiques en `PROTECT`, les endpoints de lecture seule des historiques et l'archivage empêchent la suppression en cascade des mesures/commandes.

- Les commandes ne pilotent aucun équipement physique; aucun MQTT, ESP32 ou système d’alertes n’est présent.

7. Vérifications

```
.\.venv\bin\python.exe manage.py check
.\.venv\bin\python.exe manage.py makemigrations --check --dry-run
.\.venv\bin\python.exe manage.py showmigrations
.\.venv\bin\python.exe manage.py test
```